

gem5 O3 CPU Issue Queue (IQ) 规格说明书

文档版本: 1.0

适用架构: RISC-V (gem5 O3 CPU)

参考模型: OpenC910 IssueQueue

创建日期: 2026-04-08

修订历史

版本	日期	作者	修改描述
1.0	2026-04-08	gem5 Team	初始版本, 基于 gem5 O3 CPU 实现

目录

- [术语和缩略语](#)
- [概述](#)
- [架构规格](#)
- [接口定义](#)
- [操作流程](#)
- [时序规范](#)
- [状态机定义](#)
- [性能参数](#)
- [参考资料](#)

1. 术语和缩略语

1.1 术语表

术语	英文全称	定义
IQ	Instruction Queue	指令队列, 缓存已解码等待发射的微操作
ROB	Reorder Buffer	重排序缓冲区, 保证指令按程序顺序提交
FU	Functional Unit	功能单元, 执行具体运算逻辑
FU Pool	Functional Unit Pool	功能单元池, 管理多个 FU 的分配
DynInst	Dynamic Instruction	动态指令, 携带运行时信息的指令实例
OpClass	Operation Class	操作类型, 如 IntAlu、FloatAdd、MemRead 等
SMT	Simultaneous Multi-Threading	同时多线程, 支持多线程共享 IQ 资源
DTB	Data Translation Buffer	数据转换缓冲器, 用于虚拟地址翻译

1.2 缩略语

缩写	含义
O3	Out-of-Order (乱序执行)
IEW	Issue/Execute/Writeback (发射/执行/写回级)
RF	Register File (寄存器堆)
ALU	Arithmetic Logic Unit (算术逻辑单元)
FPU	Floating Point Unit (浮点运算单元)
LSU	Load Store Unit (访存单元)

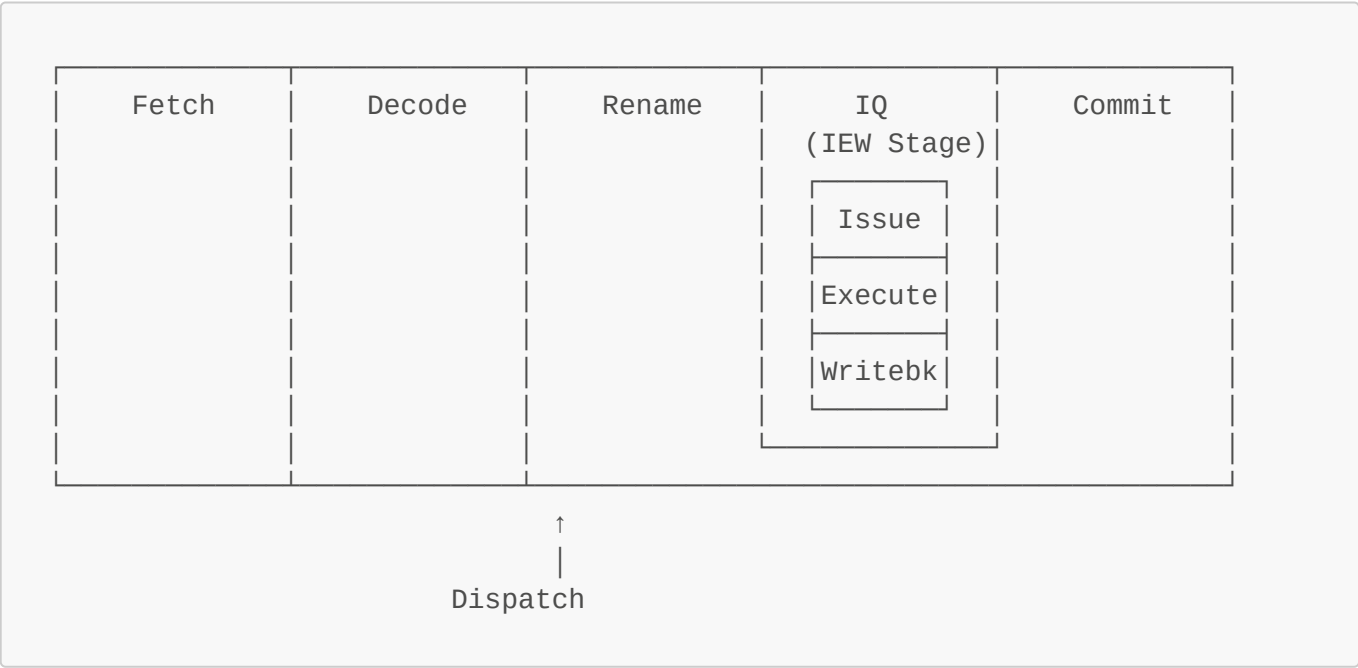
2. 概述

2.1 功能描述

Issue Queue (IQ) 是 gem5 O3 CPU 乱序执行引擎的核心组件，位于 IEW (Issue/Execute/Writeback) 级，主要功能包括：

- 1. **指令缓冲**: 缓存从 Rename 级 dispatch 过来的已重命名微操作
- 2. **依赖跟踪**: 通过 Dependency Graph 跟踪指令间的寄存器依赖关系
- 3. **就绪检测**: 当指令的所有源操作数就绪时，将其标记为可发射
- 4. **资源分配**: 根据 OpClass 分配对应的 Functional Unit
- 5. **指令发射**: 每个周期选择就绪指令发射到对应的 FU 执行
- 6. **内存排序**: 通过 MemDepUnit 保证内存操作的顺序性

2.2 在流水线中的位置



2.3 设计特点

特性	描述
----	----

特性	描述
乱序发射	指令按就绪顺序发射，不依赖程序顺序
按类分离	Ready Queue 按 OpClass 分离，快速匹配 FU
依赖图	基于物理寄存器索引的数组结构，O(1) 访问
SMT 支持	支持 Partitioned/Dynamic/Threshold 三种共享策略
内存预测	独立的 MemDepUnit 处理 Load-Store 依赖

3. 架构规格

3.1 物理参数

参数	默认值	可配置范围	描述
numEntries	64	1-1024	每个 IQUnit 的条目数量
issueWidth	8	1-16	每周期最大发射指令数
dispatchWidth	8	1-16	每周期最大 Dispatch 指令数
numIQUnits	1	1-8	IQUnit 数量 (多队列配置)
numPhysRegs	256	-	物理寄存器数量 (依赖图大小)

3.2 寄存器配置

寄存器类型	数量	描述
整数寄存器	256	通用整数物理寄存器
浮点寄存器	256	浮点物理寄存器
向量寄存器	256	向量物理寄存器 (RISC-V V 扩展)
谓词寄存器	32	向量谓词寄存器
条件码寄存器	0	条件码寄存器 (RISC-V 无专用 CC 寄存器)

3.3 数据结构

3.3.1 IQUnit

IQUnit	
_numEntries	: unsigned (64)
_freeEntries	: unsigned
maxEntries[MaxThreads]	: unsigned
count[MaxThreads]	: unsigned
iqPolicy	: SMTQueuePolicy

fuPool*	: FUPool*
---------	-----------

3.3.2 InstructionQueue

InstructionQueue	
iqs[]	: vector<IQUnit*>
instList[MaxThreads]	: list<DynInstPtr>
instsToExecute	: list<DynInstPtr>
readyInsts[Num_OpClasses]	: priority_queue<DynInstPtr>
dependGraph	: DependencyGraph
memDepUnit[MaxThreads]	: MemDepUnit
deferredMemInsts	: list<DynInstPtr>
blockedMemInsts	: list<DynInstPtr>
retryMemInsts	: list<DynInstPtr>
nonSpecInsts	: map<SeqNum, DynInstPtr>
listOrder	: list<ListOrderEntry>
regScoreboard[]	: vector<bool>

3.3.3 DependencyEntry

DependencyEntry	
inst	: DynInstPtr
next	: DependencyEntry*

3.3.4 ListOrderEntry

ListOrderEntry	
queueType	: OpClass
oldestInst	: InstSeqNum

3.4 操作类型 (OpClass)

OpClass	描述	默认 FU 数量	典型延迟
---------	----	----------	------

OpClass	描述	默认 FU 数量	典型延迟
IntAlu	整数 ALU 运算	4	1
IntMult	整数乘法	1	3-5
IntDiv	整数除法	1	20-40
FloatAdd	浮点加法	2	3-4
FloatCmp	浮点比较	2	3-4
FloatMult	浮点乘法	2	4-5
FloatDiv	浮点除法	1	10-20
FloatSqrt	浮点开方	1	20-30
FloatMisc	浮点其他	1	5-10
MemRead	内存读	3	3+
MemWrite	内存写	2	2+
SimdAdd	SIMD 加法	2	2-3
SimdMult	SIMD 乘法	2	3-4
SimdDiv	SIMD 除法	1	10-15
SimdMisc	SIMD 其他	1	5-10
Branch	分支指令	-	1
lprAccess	特权寄存器访问	1	10-50
No_OpClass	空操作/NOP	-	0

4. 接口定义

4.1 输入接口

4.1.1 Dispatch 输入

信号名	位宽	方向	描述
dispatch_ready	1	Input	Dispatch 级就绪
dispatch_inst[]	DynInst	Input	Dispatch 的指令数组
dispatch_count	8	Input	本周期 Dispatch 指令数
dispatch_valid[]	1	Input	每条指令的有效位

4.1.2 Writeback 输入

信号名	位宽	方向	描述
wb_ready	1	Input	Writeback 级就绪
wb_dest_reg[]	8	Input	写回的目标寄存器
wb_valid[]	1	Input	写回有效位
wb_exception	1	Input	异常标志

4.1.3 Commit 输入

信号名	位宽	方向	描述
commit_seqnum	64	Input	已提交的指令序列号
commit_tid	8	Input	提交指令的线程 ID
squash_request	1	Input	冲刷请求
squash_seqnum	64	Input	冲刷目标序列号

4.2 输出接口

4.2.1 Issue 输出

信号名	位宽	方向	描述
issue_ready	1	Output	IQ 就绪可发射
issue_inst[]	DynInst	Output	发射的指令数组
issue_count	8	Output	本周期发射指令数
issue_full	1	Output	IQ 满标志

4.2.2 FU 分配输出

信号名	位宽	方向	描述
fu_alloc_valid	1	Output	FU 分配有效
fu_alloc_idx	8	Output	分配的 FU 索引
fu_alloc_opclass	8	Output	操作类型
fu_busy[]	1	Output	FU 忙碌状态

4.3 配置接口 (Python)

```
# IQUnit 配置
class IQUnit(SimObject):
    type = "IQUnit"
    cxx_class = "gem5::o3::IQUnit"
```

```
cxx_header = "cpu/o3/inst_queue.hh"

numEntries = Param.Unsigned(64, "指令队列条目数")
fuPool = Param.FUPool(DefaultFUPool(), "功能单元池")
numThreads = Param.Unsigned(1, "硬件线程数")
smtIQPolicy = Param.SMTQueuePolicy("Partitioned", "SMT 共享策略")
smtIQThreshold = Param.Int(100, "SMT 阈值参数")

# BaseO3CPU IQ 相关参数
class BaseO3CPU(BaseCPU):
    # IQ 配置
    instQueues = VectorParam.IQUnit(IQUnit(), "IQ 向量")

    # 宽度参数
    fetchWidth = Param.Unsigned(8, "取指宽度")
    decodeWidth = Param.Unsigned(8, "解码宽度")
    renameWidth = Param.Unsigned(8, "重命名宽度")
    dispatchWidth = Param.Unsigned(8, "分发宽度")
    issueWidth = Param.Unsigned(8, "发射宽度")
    wbWidth = Param.Unsigned(8, "写回宽度")
    commitWidth = Param.Unsigned(8, "提交宽度")

    # 延迟参数
    issueToExecuteDelay = Param.Cycles(1, "发射到执行延迟")
    commitToIEWDelay = Param.Cycles(1, "Commit 到 IEW 延迟")
    renameToIEWDelay = Param.Cycles(2, "Rename 到 IEW 延迟")
```

5. 操作流程

5.1 指令插入流程 (Insert)



5.1.1 详细流程

```
void InstructionQueue::insert(const DynInstPtr &new_inst) {
    // 1. 添加到线程指令列表
    instList[new_inst->threadNumber].push_back(new_inst);

    // 2. 查找可用 IQUnit
    auto iq = findIQ(new_inst);
    assert(iq != nullptr);

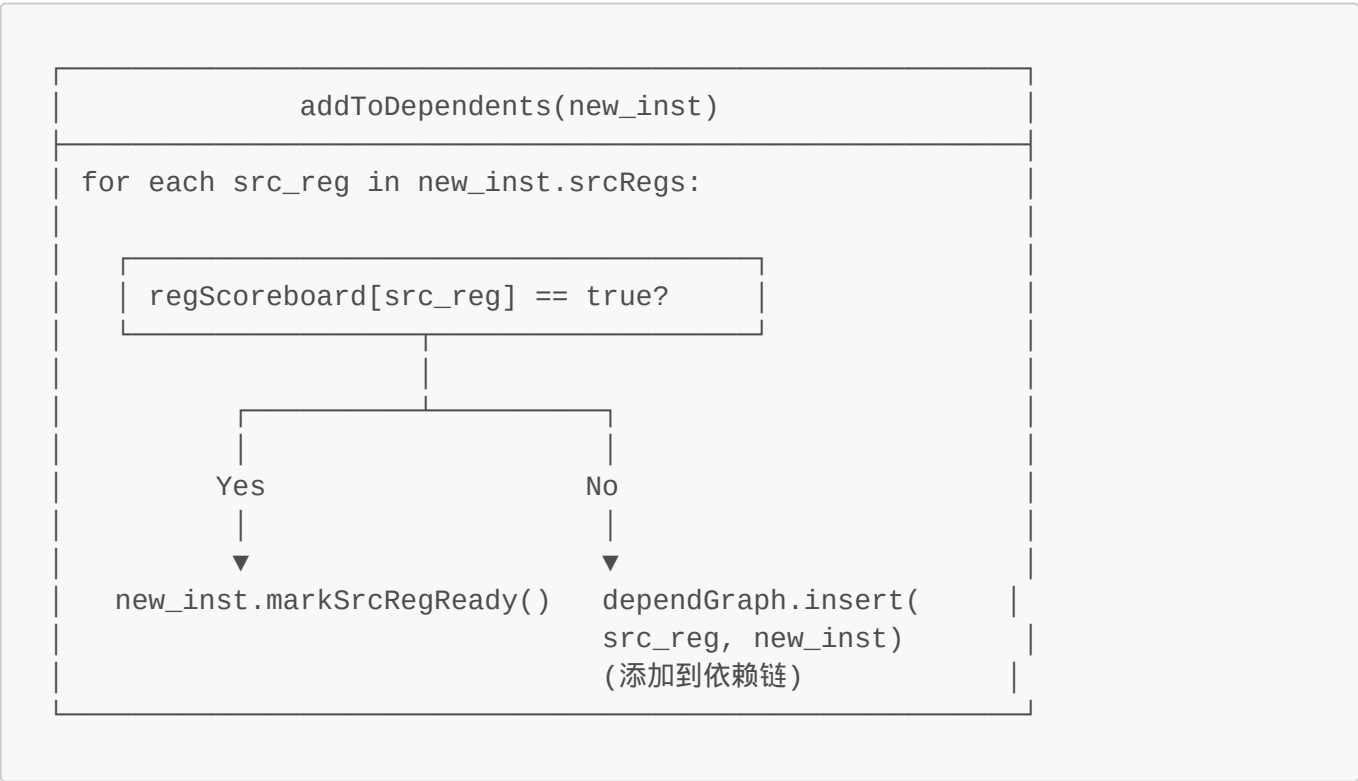
    // 3. 插入到 IQUnit
    iq->insert(new_inst);

    // 4. 建立依赖关系
    addToDependents(new_inst); // 添加为消费者
    addToProducers(new_inst); // 设置为目标寄存器生产者

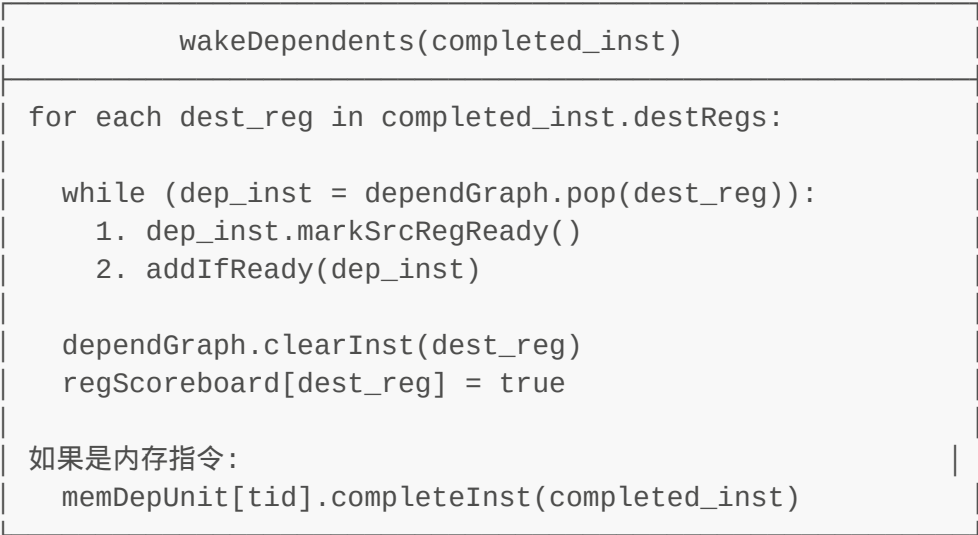
    // 5. 内存指令特殊处理
    if (new_inst->isMemRef()) {
        memDepUnit[tid].insert(new_inst);
    } else {
        addIfReady(new_inst);
    }

    iqStats.instsAdded++;
}
```

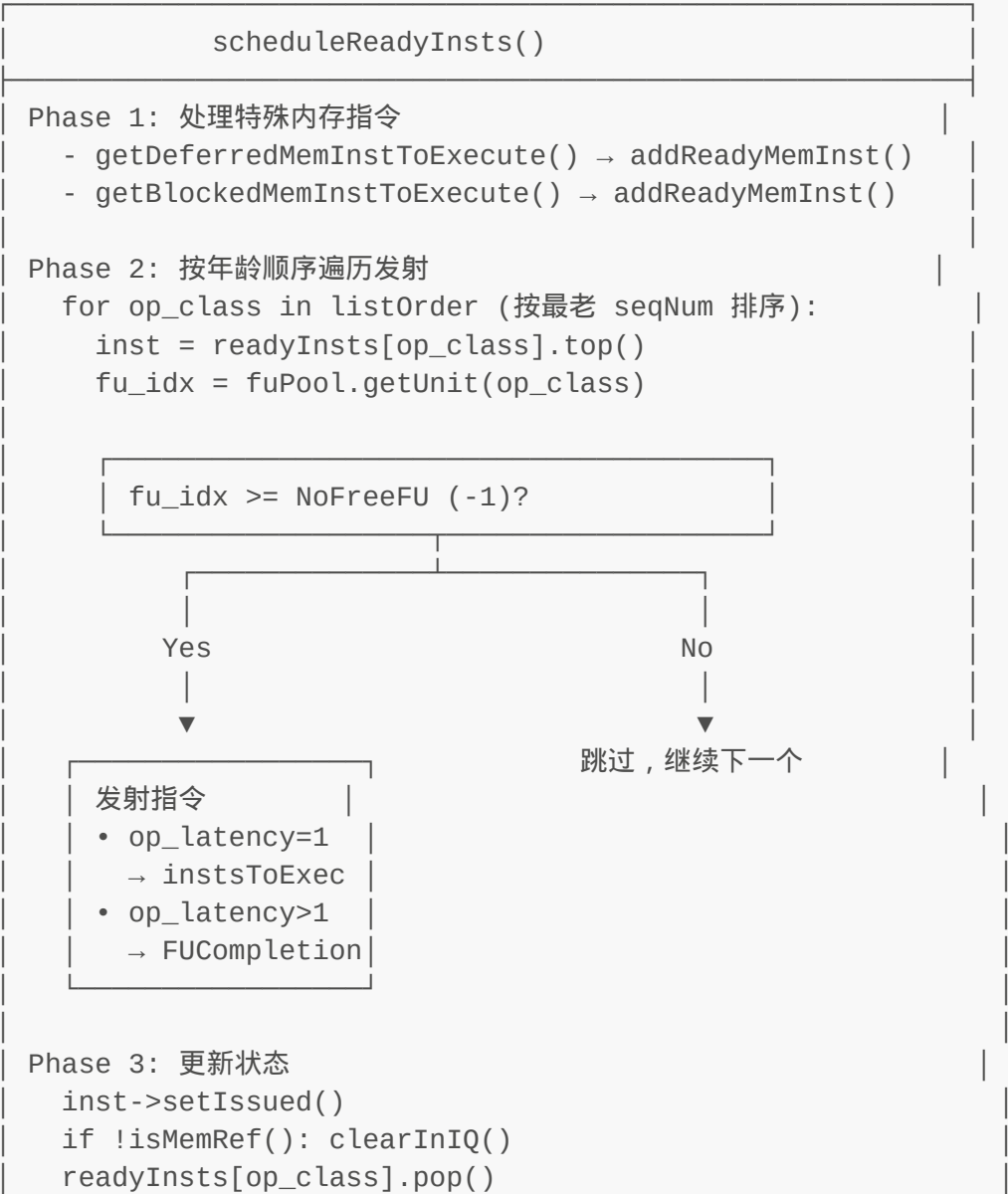
5.2 依赖添加流程



5.3 唤醒依赖流程



5.4 指令调度发射流程



moveToYoungerInst() / queueOnList=false

5.5 冲刷 (Squash) 流程

doSquash(tid)

输入 : squashedSeqNum[tid] (冲刷到该序列号)

从 instList[tid] 尾部向前遍历:
for inst in instList[tid] (逆序):
 if inst.seqNum > squashedSeqNum[tid]:

 1. 从依赖图移除
 for src_reg: dependGraph.remove()

 2. 清空依赖图头部
 for dest_reg: dependGraph.clearInst()

 3. 标记冲刷状态
 inst.setSquashedInIQ()
 inst.setIssued() ← 流向 Commit
 inst.setCanCommit()
 inst.clearInIQ()

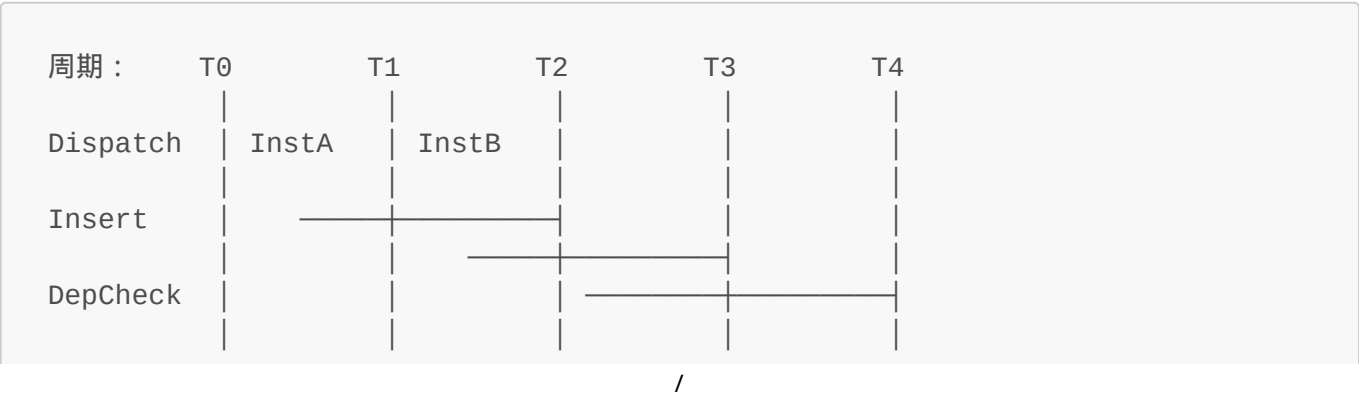
 4. 从 instList 移除
 instList[tid].erase(inst)

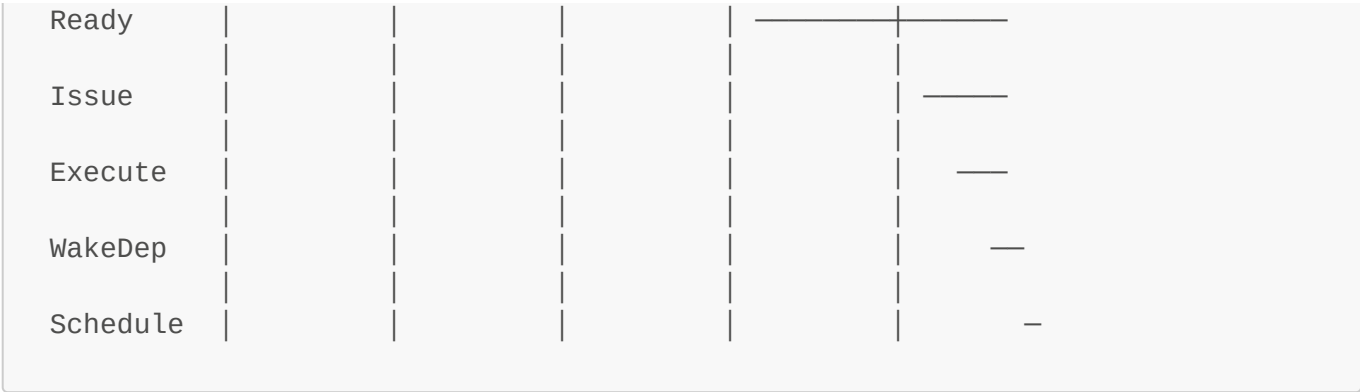
同时处理 nonSpecInsts:
 erase(nonSpecInsts[seqNum > squashedSeqNum])

通知 MemDepUnit:
 memDepUnit[tid].squash(squashedSeqNum[tid], tid)

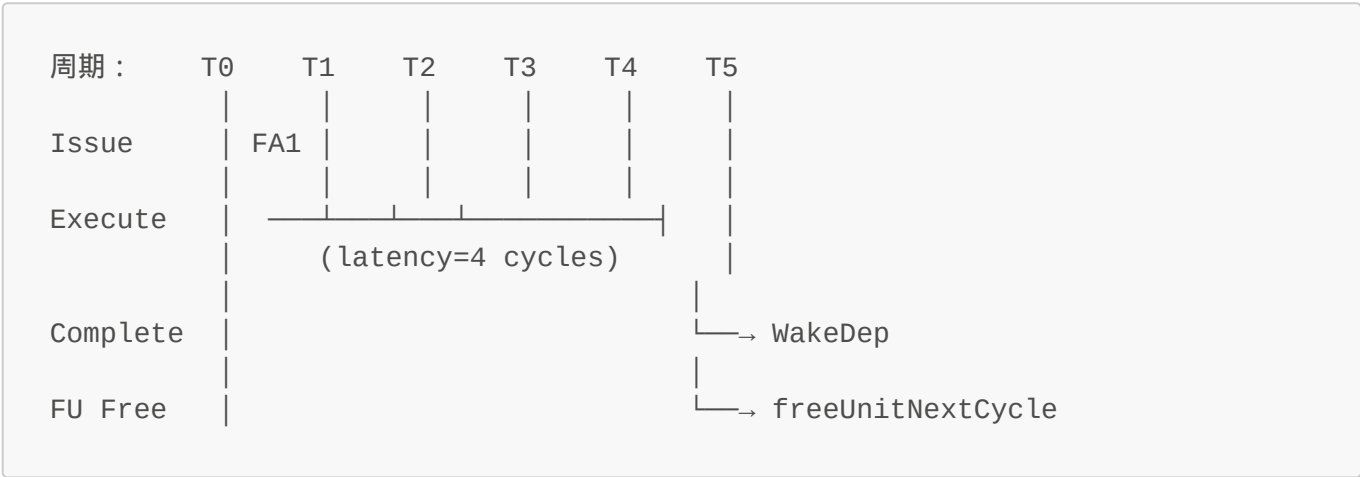
6. 时序规范

6.1 典型操作时序





6.2 多周期 FU 时序 (以 FloatAdd 为例)



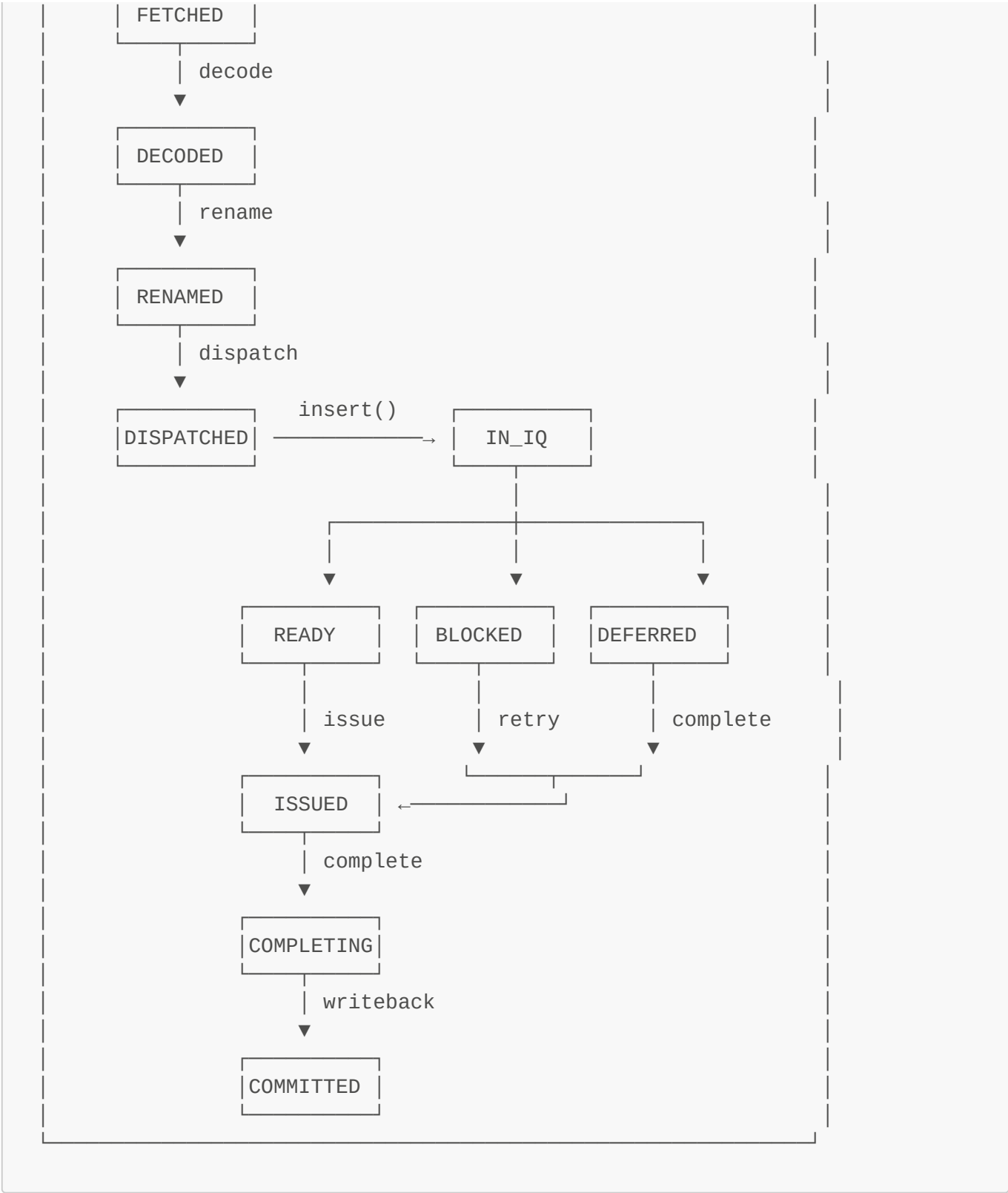
6.3 流水线延迟参数

路径	参数名	默认值 (cycles)
Fetch → Decode	fetchToDecodeDelay	1
Decode → Rename	decodeToRenameDelay	1
Rename → IEW	renameToIEWDelay	2
IEW → Commit	iewToCommitDelay	1
Commit → Fetch	commitToFetchDelay	1
Issue → Execute	issueToExecuteDelay	1
Commit → IEW	commitToIEWDelay	1

7. 状态机定义

7.1 指令状态机

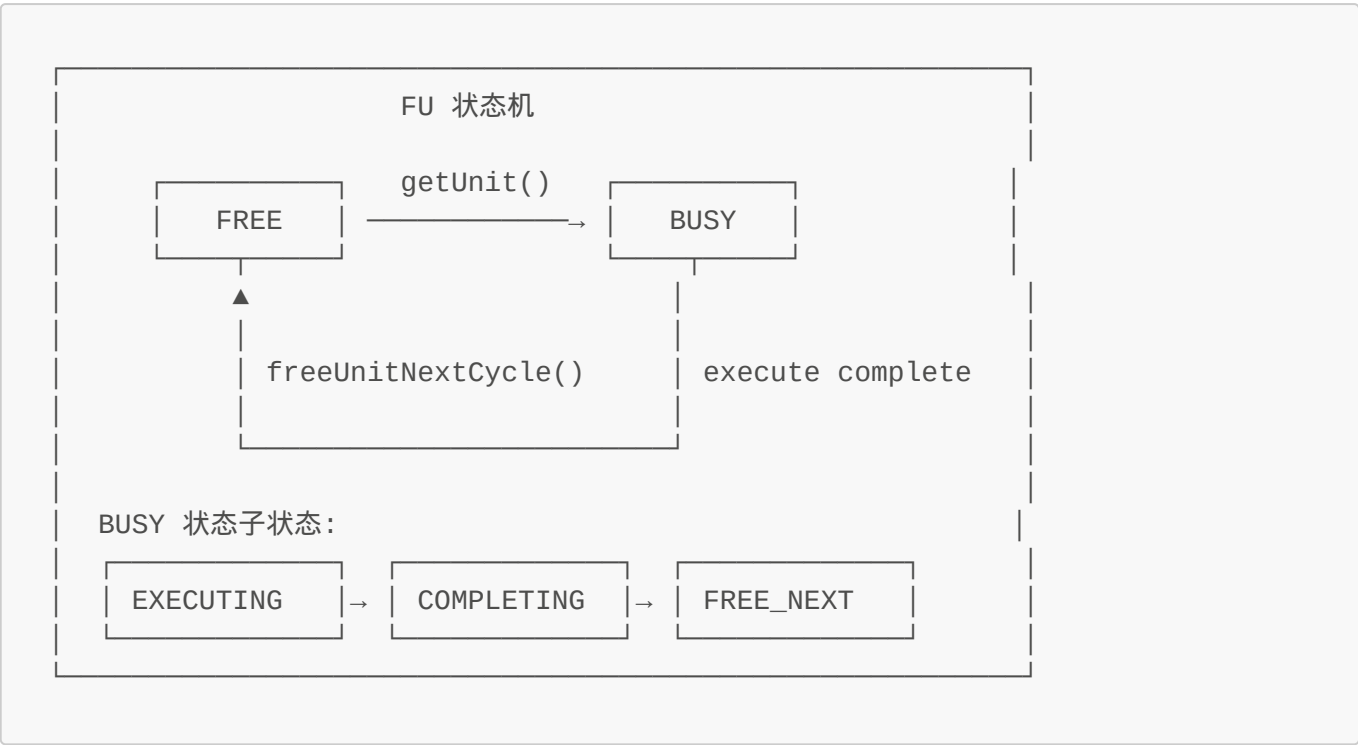




7.2 IQUnit 状态

状态	条件	动作
NORMAL	<code>_freeEntries > 0</code>	接受新指令
PARTIAL_FULL	<code>_freeEntries < threshold</code>	降低 dispatch 优先级
FULL	<code>_freeEntries = 0</code>	stall dispatch
SQUASHING	<code>squash_request = 1</code>	清除指定序列号后指令

7.3 FU 状态



8. 性能参数

8.1 默认配置性能

指标	值	说明
IQ 容量	64 entries	单 IQUnit
最大发射宽度	8 IPC	issueWidth=8
依赖图大小	800+ entries	所有物理寄存器
Ready Queue 数量	18	Num_OpClasses

8.2 统计信息 (Statistics)

统计项	类型	描述
instsAdded	Counter	添加到 IQ 的指令数
nonSpecInstsAdded	Counter	非推测指令添加数
instsIssued	Counter	发射指令总数
intInstsIssued	Counter	整数指令发射数
floatInstsIssued	Counter	浮点指令发射数
memInstsIssued	Counter	内存指令发射数
squashedInstsIssued	Counter	被冲刷的就绪指令数

统计项	类型	描述
numIssuedDist	Distribution	每周期发射数分布
issueRate	Formula	发射率 (指令/周期)
fuBusy	Vector	各线程 FU 繁忙次数
fuBusyRate	Formula	FU 繁忙率

8.3 性能优化建议

场景	建议配置	说明
高 IPC 负载	numEntries=128, issueWidth=8	增加 IQ 容量和发射宽度
内存密集	LQEntries=64, SQEntries=64	增加 load/store queue 条目
浮点密集	增加 Float FU 数量	提高浮点吞吐
多线程	smtIQPolicy=Dynamic	动态共享 IQ 资源

8.4 SMT 共享策略对比

策略	优点	缺点	适用场景
Partitioned	公平，易实现	资源利用率低	负载均衡的多线程
Dynamic	资源利用率高	可能饿死弱线程	负载不均衡场景
Threshold	平衡公平和效率	参数调优复杂	通用场景

9. 参考资料

9.1 源文件

文件路径	描述
src/cpu/o3/inst_queue.hh	IQ 头文件
src/cpu/o3/inst_queue.cc	IQ 实现
src/cpu/o3/dep_graph.hh	依赖图实现
src/cpu/o3/fu_pool.hh	FU 池实现
src/cpu/o3/mem_dep_unit.hh/cc	内存依赖单元
src/cpu/o3/IQUnit.py	IQUnit Python 配置
src/cpu/o3/BaseO3CPU.py	O3 CPU 参数定义

9.2 RISC-V 集成

文件路径	描述
------	----

文件路径	描述
src/arch/riscv/RiscvCPU.py	RISC-V CPU 定义
src/arch/riscv/RiscvISA.py	RISC-V ISA 定义
src/arch/riscv/isa.cc	ISA 实现

9.3 相关文档

- OpenC910 IssueQueue SPEC
- gem5 O3 CPU 文档: https://www.gem5.org/documentation/general_docs/o3_cpu/
- RISC-V 规范: <https://riscv.org/specifications/>

附录 A: IQ 配置示例

```
# RISC-V O3 CPU 配置示例
class RiscvO3CPU(BaseO3CPU, RiscvCPU):
    mmu = RiscvMMU()

    # IQ 配置
    instQueues = VectorParam.IQUnit(
        IQUnit(
            numEntries=64,
            fuPool=DefaultFUPool(),
            numThreads=1,
            smtIQPolicy="Partitioned",
            smtIQThreshold=100,
        ),
        size=1
    )

    # 宽度配置
    fetchWidth = 8
    decodeWidth = 8
    renameWidth = 8
    dispatchWidth = 8
    issueWidth = 8
    wbWidth = 8
    commitWidth = 8

    # 延迟配置
    issueToExecuteDelay = 1
    commitToIEWDelay = 1
    renameToIEWDelay = 2

    # 寄存器配置
    numPhysIntRegs = 256
    numPhysFloatRegs = 256
    numPhysVecRegs = 256
    numROBEntries = 192
```

```
# FU 池配置
fuPool = FUPool(
    funcUnits=[
        FuncUnit(opClasses=['IntAlu', 'IntMult', 'IntDiv']),
        FuncUnit(opClasses=['FloatAdd', 'FloatMult', 'FloatDiv']),
        FuncUnit(opClasses=['SimdAdd', 'SimdMult']),
        FuncUnit(opClasses=['MemRead', 'MemWrite']),
        FuncUnit(opClasses=['IprAccess']),
    ]
)
```

文档结束